ME2605 / DDA6204 · Neural Net Training · Fall 2026

Lecture 6: SGD

Ruoyu Sun

OPENING

Today's Plan

01 Part I

SGD & Variants

Update order
Mini-batch
Reshuffling
Momentum

02 Part II

SGD Theory

Convergence theory
Reading the rates
Why SGD is fast

03 Part III

LR Schedules

Warmup, step decay
Inverse sqrt, linear
Cosine, WSD
What LLMs use

04 Part IV

From SGD to Adam

Averaging, momentum
AdaGrad, RMSProp
Adam and variants

Learning Goals

01 Name the three design choices in SGD, and say what each one changes.

02 State when SGD converges, for diminishing and for constant stepsizes.

03 Tell which rate a run follows, from the time it takes to cut the error 10×.

04 Explain why SGD is fast, and when memory forces the batch size.

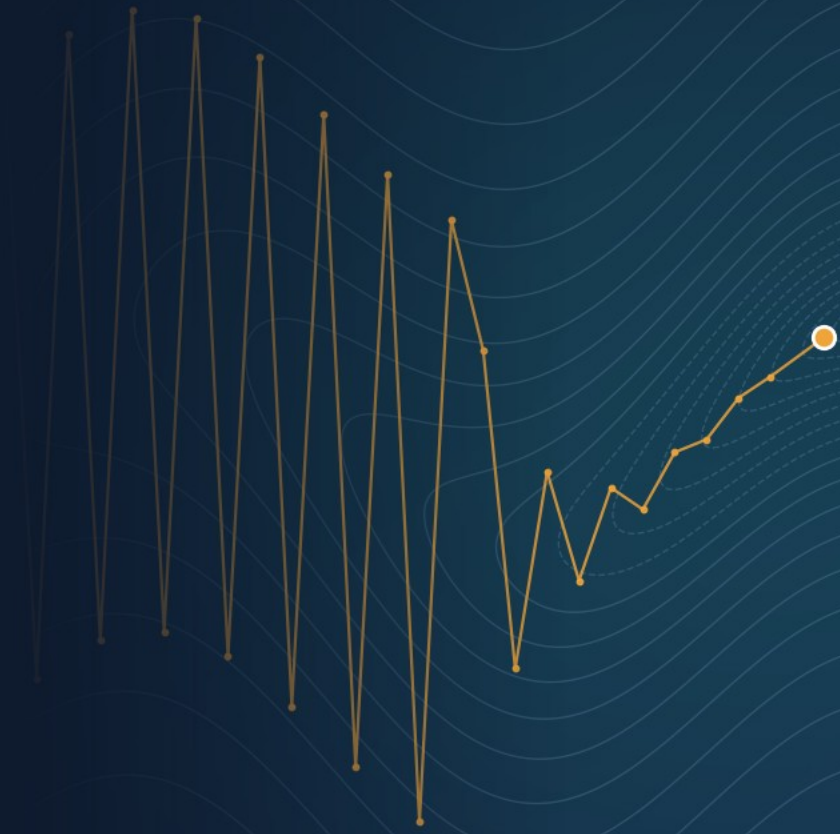
05 Choose a learning-rate schedule, and say what each one changes.

06 Derive Adam from momentum and a diagonal preconditioner.

PART I

SGD and Its Practical Variants

Update order, mini-batch, reshuffling, momentum



I SGD & Variants II SGD Theory III LR Schedules IV From SGD to Adam

SGD for Finite Sum

- Consider an optimization problem

$$\min_{w\in Rd} F(w) \triangleq \sum_{i=1}^{n} F_i(w),$$

where F_i is differentiable.

- **SGD** (Stochastic Gradient Method): at each iteration t, pick a sample $i_t \in [n]$ **uniformly at random**, then update

$$w_{t+1} = w_t - \alpha_t \nabla F_{it}(w^t).$$

- **Remark**: SGD is not necessarily a descent method; so in optimization field, people call SG method. But following convention in machine learning, we call SGD in the lecture.

Example of Finite-Sum: Supervised Learning Loss in Machine Learning

We have seen linear regression: $\min_w \sum_i (a_i^T w - b_i)^2$.

In machine learning, a common problem is

$$\min_w \sum_i \ell(y_i, f_w(x_i)),$$

where l is a loss function, and $f_w(x)$ is a neural network.

Can set $F_i(w) = \ell(y_i, f_w(x_i))$, the loss for **one sample**.

More commonly, set $F_i(w)$ to be the loss for a **mini-batch of samples**.

Variants of SGD for Finite Sum

Three design choices, one slide each: **update order**, **block size**, **update direction**.

1 Update order

Which sample at which step?

cyclic · random permutation · independent randomization

2 Block size

How many samples per step?

mini-batch SGD: average the gradient over a batch

3 Update direction

A better direction than the pure gradient?

using momentum: $w - w_{\text{old}}$

Stepsize? Not that straightforward — will discuss later.

Update Order: Which Sample at Each Step?

Example with $n = 3$ samples. Each square = one SGD step using sample i.



In practice: reshuffling every epoch (random permutation) is the default.
Most theory analyzes independent randomization.

Block Size: Mini-batch SGD

- **Block size**: **mini-batch SGD**. Pick a mini-batch of samples B_i at each iteration t, and update

$$w \leftarrow w - \alpha_t \sum_{k \in Bi} \nabla F_k(w).$$

One epoch with $n = 12$ samples and batch size $|B| = 3$ (after shuffling):



|B| = 1
plain SGD: cheap, noisy

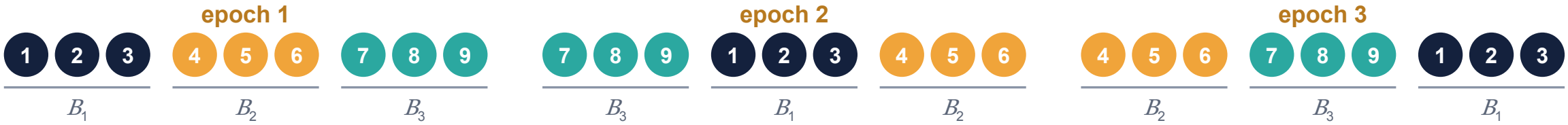
mini-batch: in between; one epoch = n/|B| updates

|B| = n
full GD: exact, expensive

Reshuffling for Mini-batch SGD

n = 9 samples, batch size $|B|$ = 3. Color = the batch a sample belongs to in epoch 1.

Option 1 · Batch-level shuffling fixed batches; only their order is reshuffled



Option 2 · Sample-level random reshuffling reshuffle all samples, then cut into new batches



- **Option 1**: the same mini-batch gradients every epoch, only in a different order.
- **Option 2**: new batches every epoch → more randomness — this is **random reshuffling**, the default of PyTorch `DataLoader(shuffle=True)`.

Random Reshuffling vs. i.i.d. Sampling in Practice

In practice, random reshuffling has **two advantages** over i.i.d. sampling.

① More uniform sampling

i.i.d. sampling (with replacement)

`17, 3, 17, 25, 8, 3, …`

- ≈ 37% of samples never used in an epoch: $(1 - 1/n)^n \approx 1/e$
- ≈ 26% used twice or more.

Random reshuffling (without replacement)

`7, 19, 2, 25, 11, 4, …`

- every sample used **exactly once** per epoch.

② Less sampling time on hardware

Example: Jellyfish [Recht & Ré, 2013], SGD on the Netflix data: the shuffle phase with replacement took almost 20 s — **significantly longer** than without replacement.

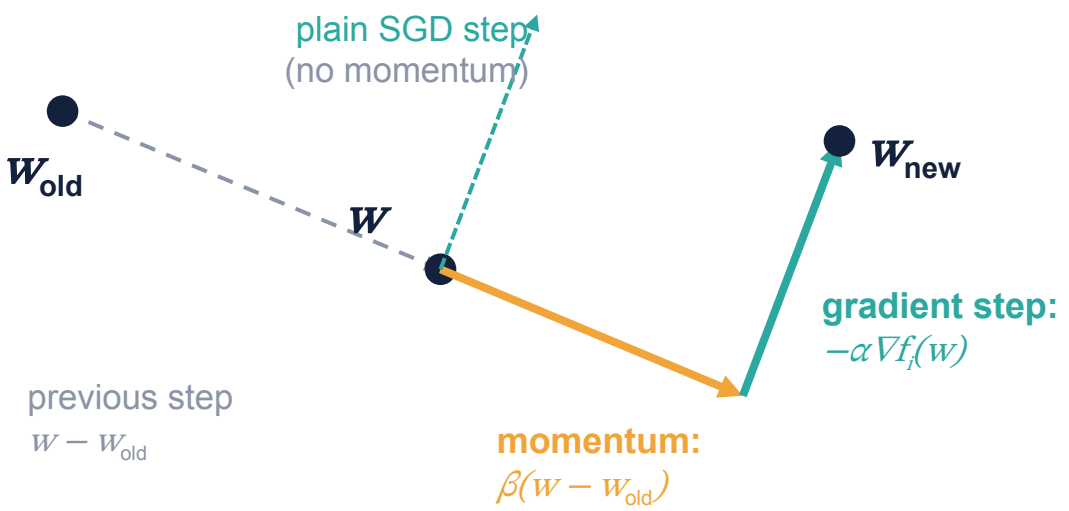
Remark: this is matrix factorization (matrix completion), not a neural net, but the setting is similar.

For neural nets we found no published measurement; feel free to look it up or compare yourself.

SGD with Momentum: How the Update Works

$$w_{t+1} = w_t - \alpha_t \nabla f_{i_t}(w_t) + \beta(w_t - w_{t-1})$$

For simplicity, write $w_t = w$, $w_{t-1} = w_{old}$: $w \leftarrow w - \alpha \nabla f_i(w) + \beta(w - w_{old})$



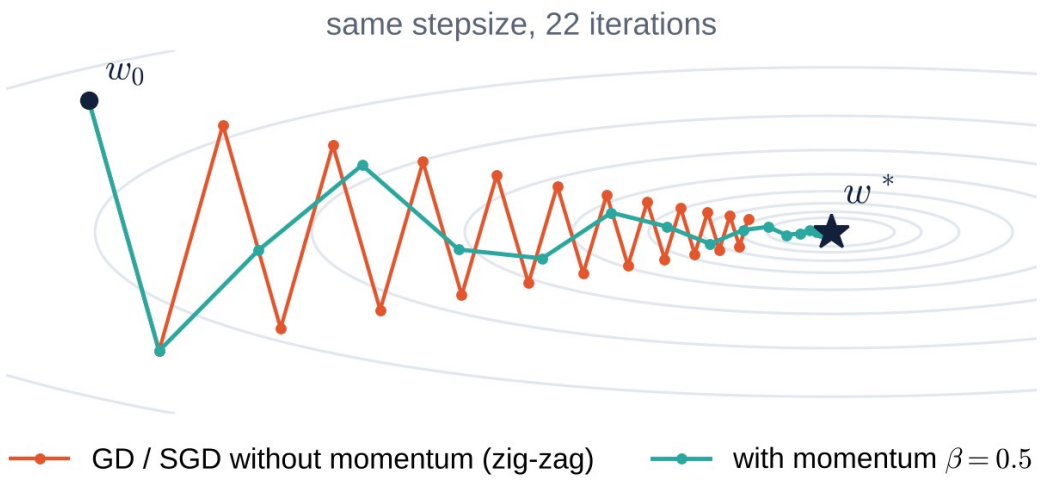
In practice, each iteration:

1. Keep the previous iterate w_{old} in memory
2. Compute a stochastic gradient $\nabla f_i(w)$
3. $w_{new} = w - \alpha \nabla f_i(w) + \beta(w - w_{old})$
4. Shift: $w_{old} \leftarrow w$, $w \leftarrow w_{new}$

$\beta = 0$: plain SGD. Common choice: $\beta = 0.9$

Idea: keep part of the previous step, then add the new gradient step.

Why Momentum Helps



1 The problem

The steep direction limits the stepsize.

Iterates **zig-zag across the valley**; progress along it is slow.

2 Momentum

Adds $\beta(w - w_{\text{old}})$.

Across the valley, consecutive steps alternate in sign and cancel → **less oscillation**.

3 Result

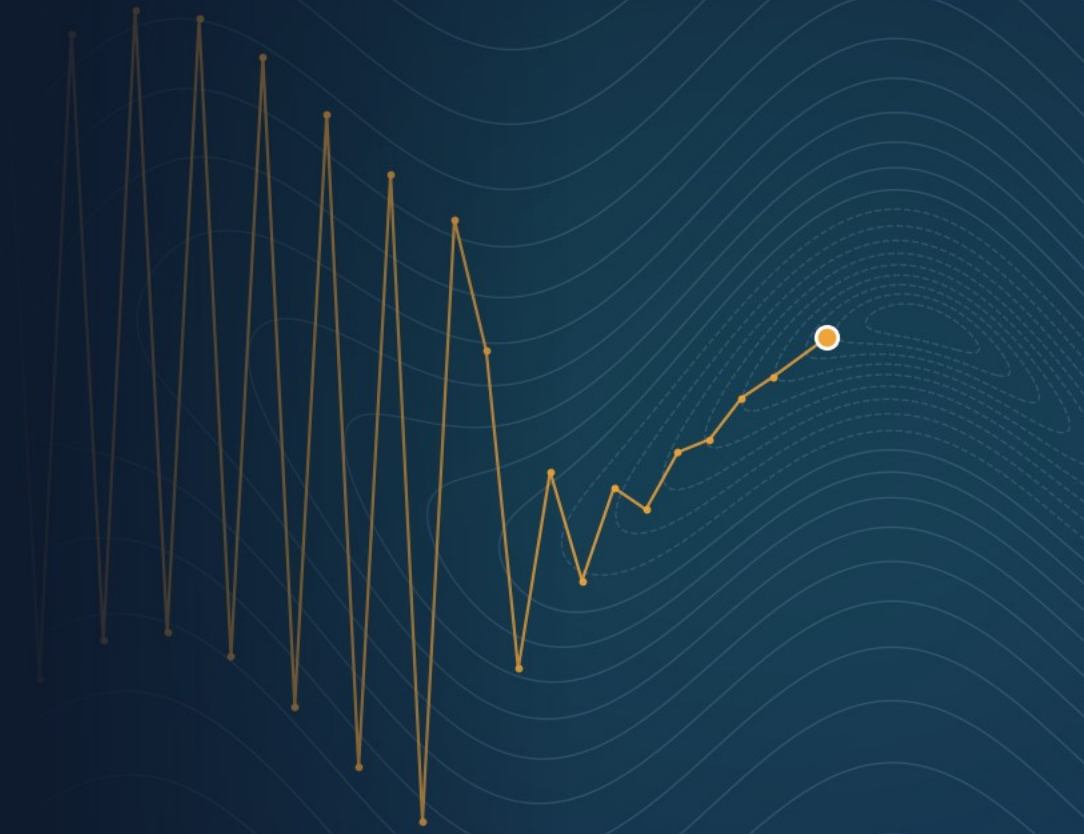
Along the valley, the steps add up → **faster progress** toward w^*.

Takeaway: momentum damps the oscillation — and that is what makes the progress faster.

PART II

SGD Theory

Convergence, reading the rates — and why SGD is fast



I SGD & Variants II SGD Theory III LR Schedules IV From SGD to Adam

Part II Roadmap

II.1 **Convergence Theory**
stationary points · diminishing vs. constant stepsize · two zones

II.2 **Reading Convergence Rates**
iteration complexity · the 10× test · experiments

II.3 **Why Is SGD Fast?**
cost saving · newer information · memory

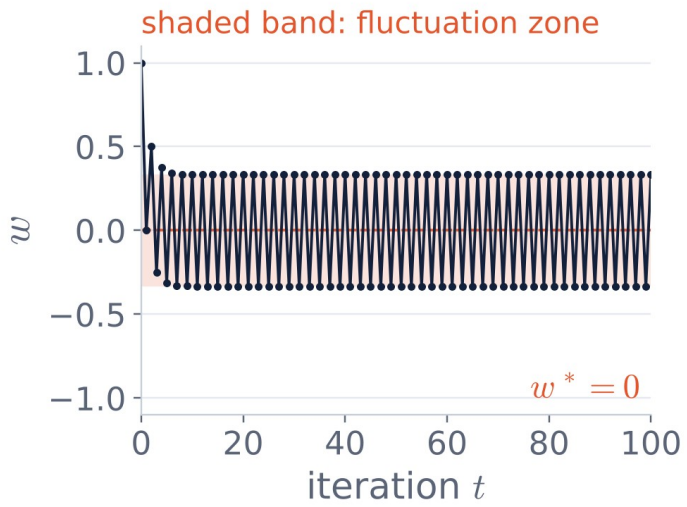
A Question

Question: Does SGD converge to stationary points?

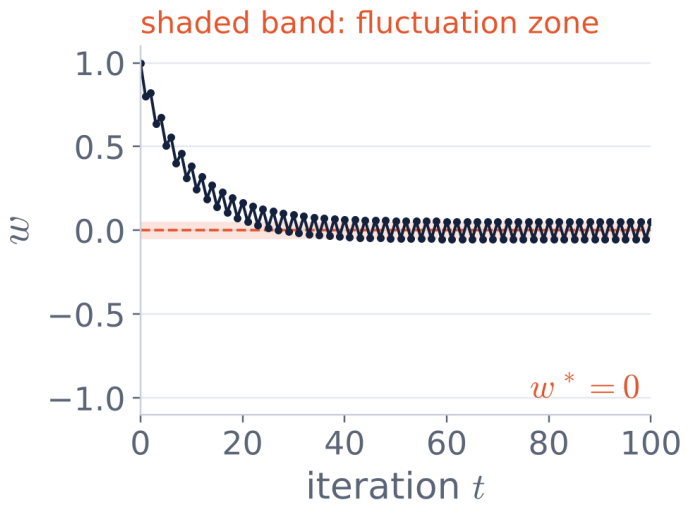
— with stepsize $\alpha < 2/L$, or a certain constant (as for GD)

SGD for Toy Example: Constant Stepsize

Solving $\min_{w \in R} 1/2(w-1)^2+1/2(w+1)^2$ by SGD: $w^0=1$, components visited in cyclic order, 100 iterations.



Left: $\alpha_t=0.5$

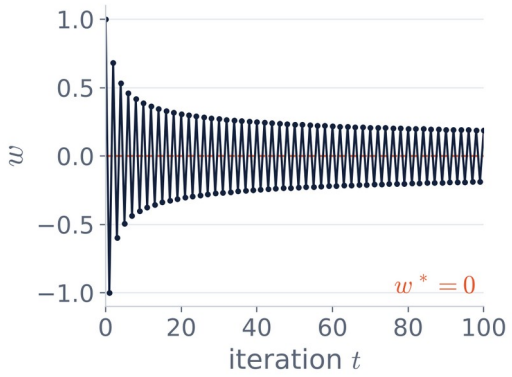


Right: $\alpha_t=0.1$

With a constant stepsize, SGD does not converge: the iterates oscillate between $\pm\alpha/(2-\alpha)$, so the fluctuation zone is proportional to the stepsize.

SGD for Toy Example: Diminishing Stepsize

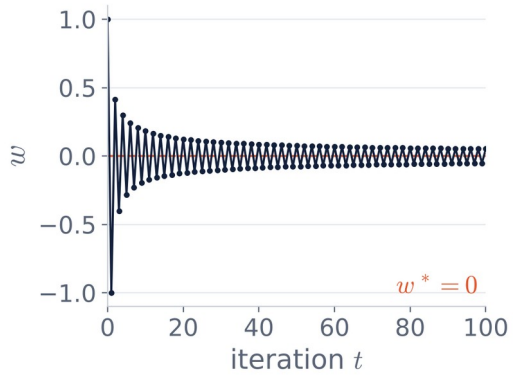
Solving $\min_{w \in R} 1/2(w-1)^2+1/2(w+1)^2$ by SGD: $w^0=1$, components visited in cyclic order, 100 iterations.



$\alpha_t=1/t^{0.25}$

Too slow

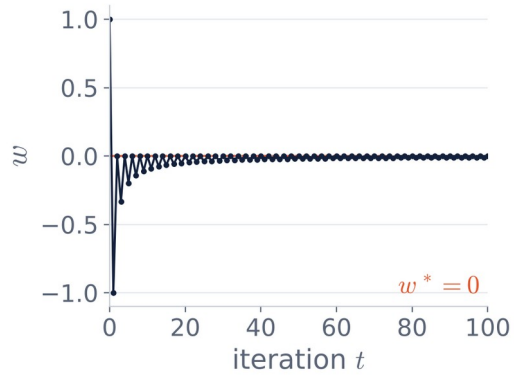
Like a **constant stepsize**: does **not seem to converge**.



$\alpha_t=1/\sqrt{t}$

Converges

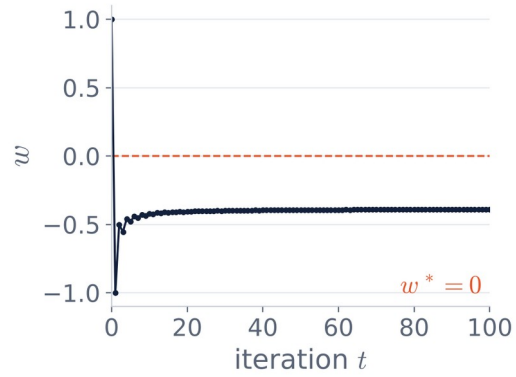
Fluctuation **shrinks**; approaches $w^*=0$.



$\alpha_t=1/t$

Converges

Dies out quickly; **converges** to $w^*=0$.



$\alpha_t=1/t^2$

Too fast

Stuck far from $w^*=0 \rightarrow$ a **non-stationary point!**

Lesson: α_t must decay, but not too fast: $\sum \alpha_t = \infty$ and $\sum \alpha_t^2 < \infty$ (Assumption 4, next slide)

Convergence Result (Non-convex)

Assumption 3 (gradient bound) For some constant $C,D>0$,

$$\|\nabla f_i(x)\| \le C + D\|f(x)\|, \forall i.$$

Assumption 4 (stepsize) The stepsize satisfies

$$\sum_{t=1}^{\infty} \alpha_t = \infty, \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty. \qquad (4)$$

e.g. $1/t^{\beta}$ for $0.5<\beta\le 1$

Theorem 1 Suppose we use $w^{t+1}=w^t-\alpha_t \nabla F_{it}(w^t)$ to solve $\sum_{i=1}^{n} F_i(w)$. Assume F is L-smooth and lower bounded. Under Assumption 3,4, for either of the following two update rule of picking i_t:

- cyclic rule,
- random permutation rule,

We have $\lim_{t\to\infty} \nabla F(w_t)=0$.

Question: can we extend the result to i.i.d. randomization?

Remark: this is the only result here that covers several update orders; the later convergence-speed results only apply to i.i.d. random sampling.

About This Result

Diminishing stepsize
$\Sigma\alpha_t = \infty,\ \Sigma\alpha_t^2 < \infty$ (4)

+

Lipschitz gradient
F is L-smooth

+

Bounded gradient error
error / variance of the stochastic gradient is bounded

↓

SGD converges to **stationary points**: every limit point $\bar{w}$ satisfies $\nabla F(\bar{w}) = 0$

- One of the **most general** convergence results for SGD; similar results hold for the randomized rule.

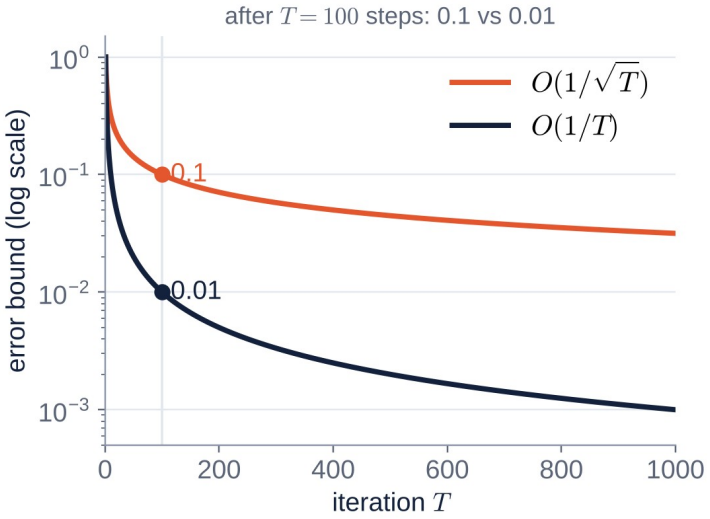
However: the Lipschitz gradient assumption may not hold in practice (similar to GD).

Diminishing Stepsize: Convergence Rates

Theorem (nonconvex, diminishing stepsize)

F is L-smooth and bounded below; stochastic gradients are unbiased with variance $\le \sigma^2$. Let $\alpha_t = \alpha_0/\sqrt{t}$ with $\alpha_0 \le 1/L$. Then

$$\min_{1\le t\le T} E\|\nabla F(w_t)\|^2 \le \frac{2(F(w_1) - F^*)}{\alpha_0\sqrt{T}} + L\sigma^2\alpha_0 \frac{1+\log T}{\sqrt{T}} = O\left(\frac{\log T}{\sqrt{T}}\right)$$



Strongly convex ($\alpha_t = c/t$, c large enough): $E[F(w_T)] - F^* = O(1/T)$
[Bottou, Curtis & Nocedal 2018, Thm 4.7]

Reading a rate: error $O(1/T) \Leftrightarrow \sim 1/\varepsilon$ iterations; error $O(1/\sqrt{T}) \Leftrightarrow \sim 1/\varepsilon^2$ iterations

Constant Stepsize SGD: Two Zones

For constant step-size, SG makes progress until variance is too big.

This leads to **two phases** when we use a constant step-size:

Graph Illustration: Multiple quadratic functions.

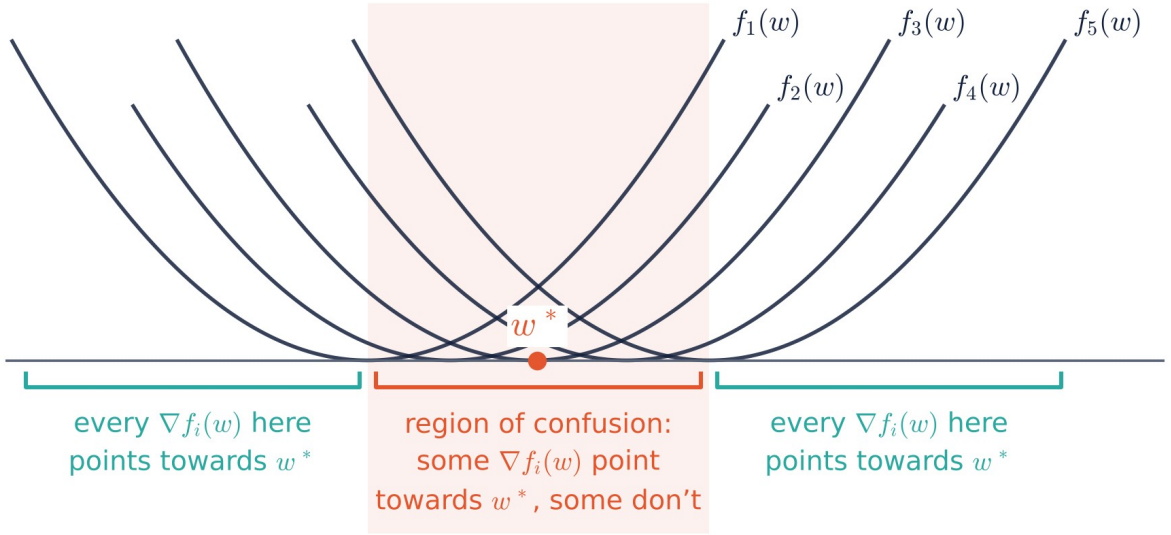


Figure: adapted from Mark Schmidt's course slides

1. **fast convergence** zone: Rapid progress when far away

2. **confusion** zone: when close, erratic behaviour (Radius of ball is proportional to the step-size.)

Constant Stepsize SGD: Graph Illustration

2-D example: fast convergence zone and confusion zone

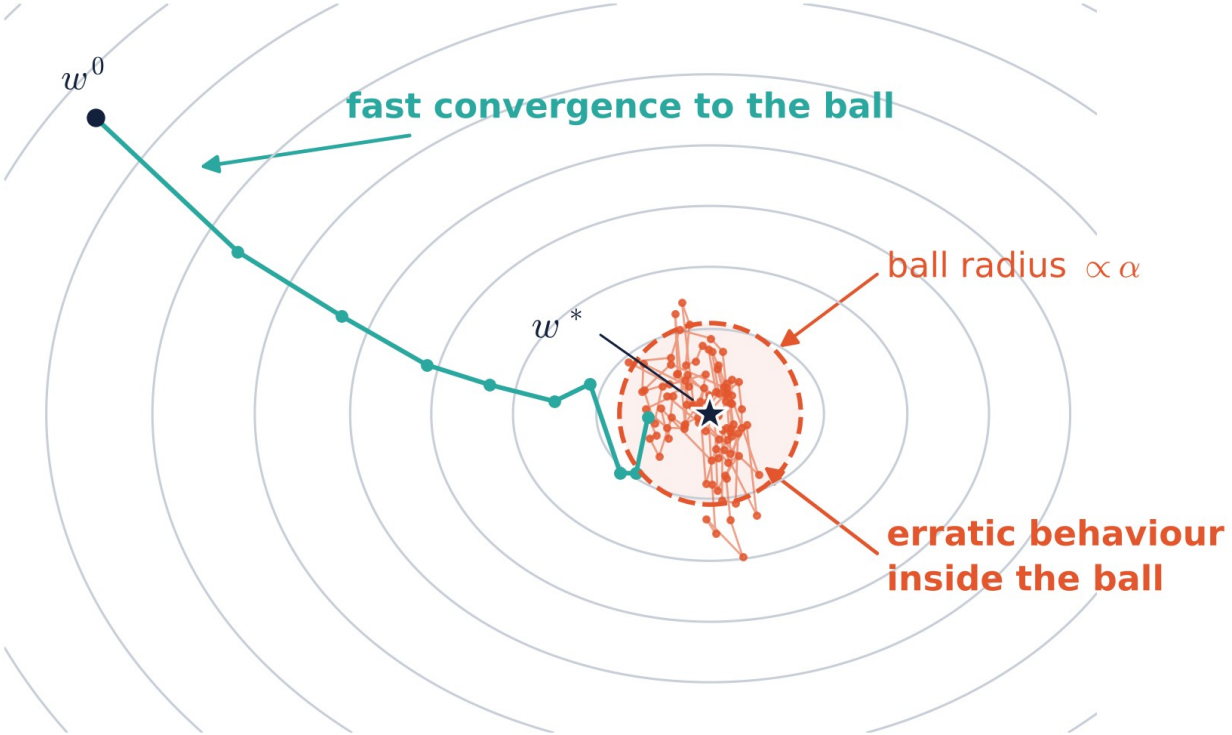


Figure: adapted from Mark Schmidt's course slides

Constant Stepsize SGD on SC Functions

Consider SGD defined as follows:

$$w_{k+1} = w_k - \alpha \nabla F(w_k; \xi_k), \qquad (1)$$

where ξ_k is a random variable independent of previous $\xi_0, \ldots, \xi_{k-1}$.

Assumption 1 (unbiased)
$\mathrm{E}[\nabla F(w_k, \xi_k)] = \nabla F(w_k), \forall k.$

Assumption 2 (bounded variance) $\mathrm{E}\|\nabla F(w_k, \xi_k) - \nabla F(w_k)\|^2 \leq \sigma^2.$

Theorem 2: For any L-smooth and μ-convex function F. Assume Assumption 1 and 2 hold, and the stepsize $\alpha \leq 2/(\mu + L)$. Let $\Phi_t = \mathrm{E}\|w_t - w^*\|^2$ and $\Phi^* = (\alpha\sigma^2)/(\mu(2 + \mu\alpha))$. Then

$$\Phi_t \leq \Phi^* + (1 - \mu\alpha)^{2t}[\Phi_0 + \alpha^2\sigma^2 - \Phi^*] \qquad (2)$$

Remark 1: As $t \to \infty$, $\mathrm{E}\|w_t - w^*\|^2$ converges to $\Phi^* = O(\sigma^2)$.

Remark 2: The size of the confusion zone **depends on the stepsize**; so if we want convergence to a single point, we need stepsize $\alpha \to 0$.

[Bottou, Curtis & Nocedal, SIAM Review 2018, Thm 4.6] states the result for function values.

Proof of Theorem 2: see Appendix

Reading the Bound of Theorem 2

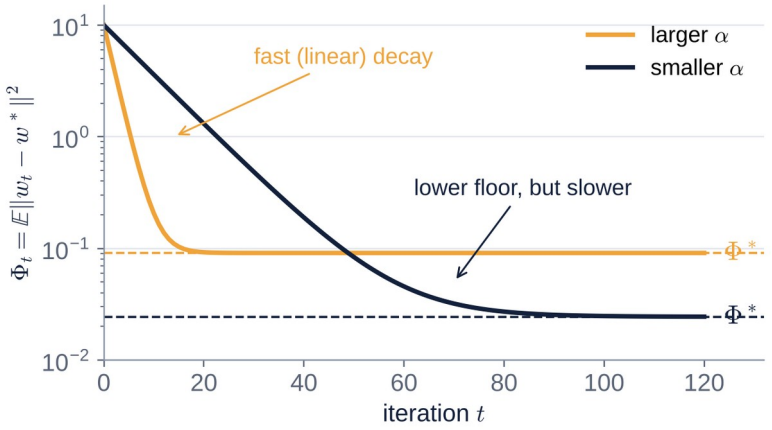
$$\Phi_t \leq \Phi^* + (1-\mu\alpha)^{2t} \cdot [\Phi_0 + \alpha^2\sigma^2 - \Phi^*]$$

distance to optimum
expected squared distance $E\|w_t - w^*\|^2$

noise floor
$\Phi^* = O(\alpha\sigma^2)$: size of the confusion zone; does **not** vanish as $t \to \infty$

geometric decay
linear convergence: shrinks by a factor $(1-\mu\alpha)^2$ per step; faster for larger α

initial gap
how far we start above the floor; forgotten exponentially fast



- t **small**: the decaying term dominates → **fast convergence zone**
- t **large**: only Φ^* remains → **confusion zone**, size $\propto \alpha$
- Larger α: faster decay, higher floor. Smaller α: lower floor, slower.

So: start with a large α, then decrease it — this is why we need LR schedules (Part III).

Lab: SGD Noise Lab

Train small and large networks on MNIST in your browser. Vary the stepsize and batch size, and check: does constant-stepsize SGD converge, or settle at a noise floor?

Open the lab →

https://ruoyus.github.io/me2605/demos/sgd-noise-lab/

scan to open on your phone

Part II Roadmap

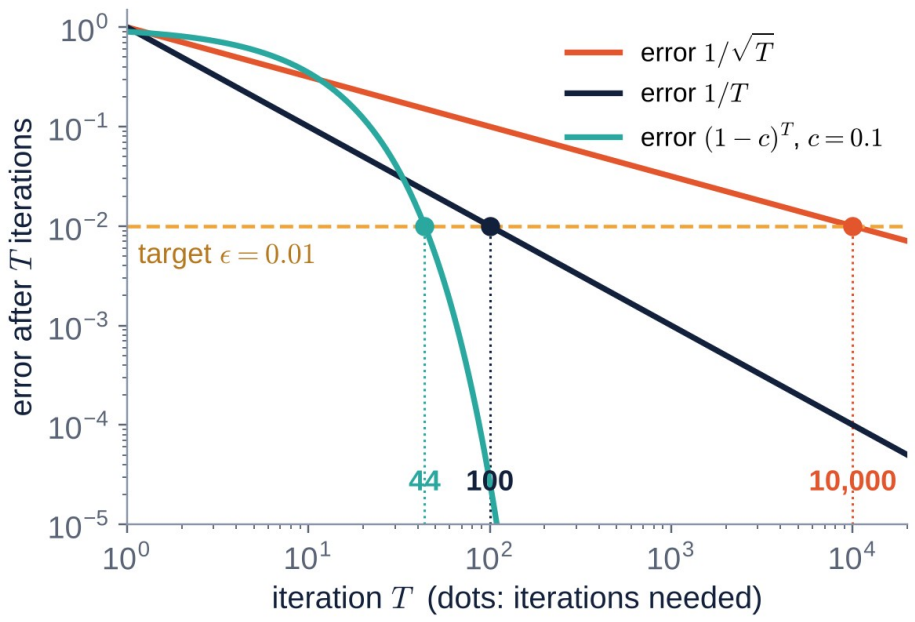
II.1 **Convergence Theory**
stationary points · diminishing vs. constant stepsize · two zones

II.2 **Reading Convergence Rates**
iteration complexity · the 10× test · experiments

II.3 **Why Is SGD Fast?**
cost saving · newer information · memory

Reading the Rates: Error vs. Iterations Needed

To understand convergence rates, we show **ideal loss plots**.



ε-accuracy: error ≤ ε. **Iteration complexity** = number of iterations T needed to reach it.

error after T steps	iterations to reach ε	ε = 0.01	where we see it
$1/\sqrt{T}$	O(1/ε²)	10,000	SGD, nonconvex / convex
1/T	O(1/ε)	100	SGD, strongly convex, α ∝ 1/t
$(1-c)^T$	O(log(1/ε))	44	GD (SC); constant-α SGD (until noise floor)

Geometric row: c = 0.1, i.e. error $(0.9)^T$ → 44 steps to reach 0.01

Log–log axes: the x-axis is log T; each rate T^{-p} is a straight line

First reading: count the iterations needed for ε = 0.01 — 44 (geometric) vs. 100 (1/T) vs. 10,000.

What Is the Rate in Practice? A First Attempt

First thought: plug numbers into the bound and compare with what we observe.

1. Estimate the constants: $L, \mu, \sigma^2, \|w_0 - w^*\|$ (or $F(w_0) - F^*$)
2. Plug them into the bound → **predicted** error after T steps
3. Run T steps and compare with the **observed** error

Challenges

Constants are hard to compute: for a neural net we do not know L or σ^2, and w^*, F^* are unknown.

The bound is worst-case: it is pessimistic — often off by orders of magnitude.

So: comparing absolute numbers tells us little. Next — a test that needs none of these constants.

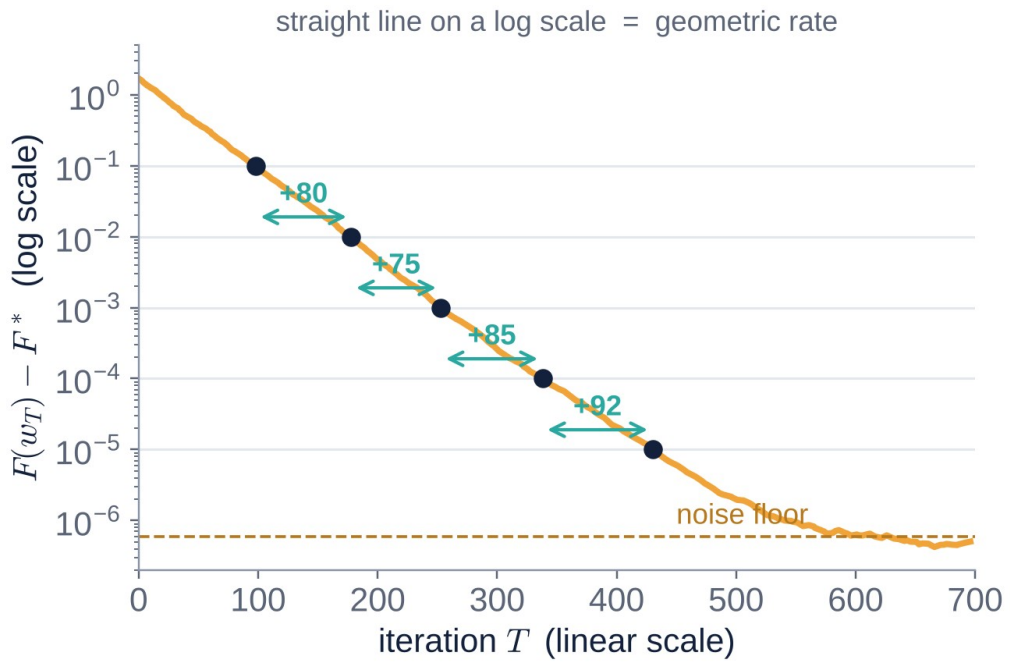
A Better Test: Time to Reduce the Error by 10×

Measure $T(\varepsilon)$ = first iteration with error ≤ ε. If $T(\varepsilon) \approx C / \varepsilon^p$, then $T(\varepsilon/10) / T(\varepsilon) = 10^p$ — the unknown constant C **cancels!**

error after T steps	iterations T(ε)	$T(10^{-3}) / T(10^{-2})$	$T(10^{-4}) / T(10^{-3})$
$1/\sqrt{T}$	C/ε^2	100×	100×
$1/T$	C/ε	10×	10×
$1/T^2$	$C/\sqrt{\varepsilon}$	$\sqrt{10} \approx 3.2\times$	$\sqrt{10} \approx 3.2\times$
$(1 - c)^T$	$\log(1/\varepsilon)/c$	$+ \ln 10/c$ steps	$+ \ln 10/c$ steps

Recipe: record when the error first hits 10^{-2}, 10^{-3}, 10^{-4}; the ratio reveals the rate — no L, σ^2, or w^* needed.

Experiment A: What Rate Does Constant α Show?



Random 10-dim least squares, well-conditioned, small label noise; constant $\alpha = 0.02$; mean of 20 runs.

error reaches	iteration T	extra steps
10^{-1}	98	—
10^{-2}	178	+80
10^{-3}	253	+75
10^{-4}	338	+85
10^{-5}	430	+92

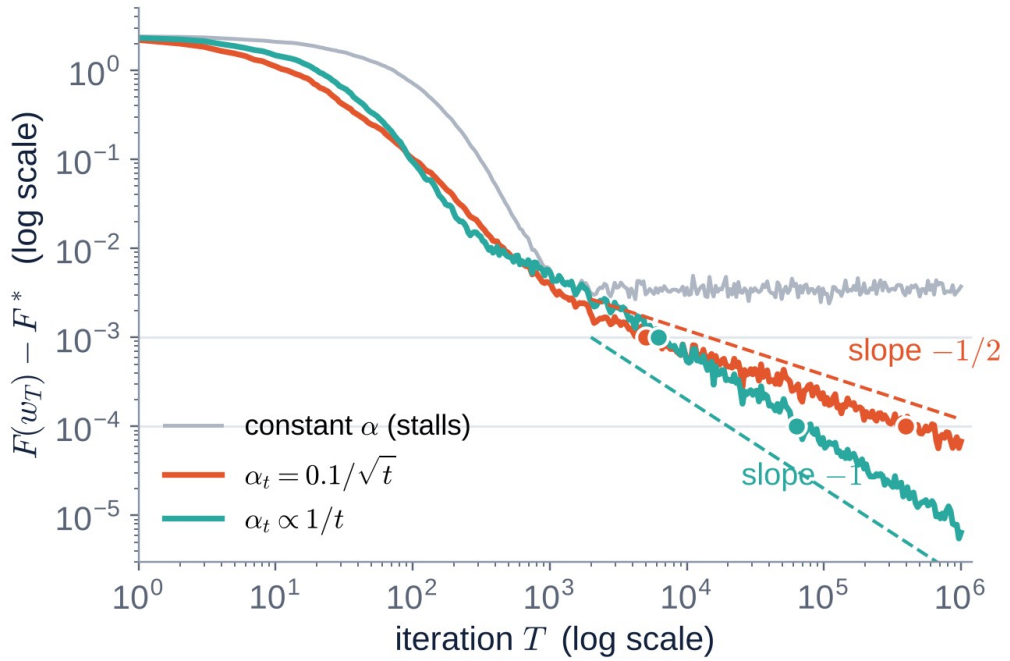
Same extra steps per 10× ⇒ geometric

theory: $\ln 10 / (2\alpha\mu) \approx 95$ steps

Message: if it takes a nearly **constant number of extra steps** to go from ε to $\varepsilon/10$ for any ε, the training shows **geometric convergence**.

e.g. error ~ $(1 - 1/\kappa)^t$ ⇒ extra steps per 10× ≈ $\kappa \ln 10 \approx 2.3\kappa$

Experiment B: Measuring Sublinear Rates



Same kind of problem, but noisier labels (so constant α stalls early, gray). Mean of 20 runs.

stepsize	T(10^{-3})	T(10^{-4})	ratio	rate
$\alpha_t \propto 1/t$	6,158	62,660	**10×**	$1/T$
$\alpha_t \propto 1/\sqrt{t}$	5,003	392,631	**78×**	$1/\sqrt{T}$

Ratio ≈ 10× ⇒ error ∝ $1/T$
Ratio ≈ 100× ⇒ error ∝ $1/\sqrt{T}$ (78× here)
Log-log plot: straight lines, slopes −1 and −1/2.

Axes: x = log T, y = log error → $error \propto T^{-p}$ is a line of slope −p

Message: if it takes **10× extra steps** to go from ε to $\varepsilon/10$ for any ε, the training shows a $1/T$ **rate.**

Discussions on Measuring Speed

1. The previous slides show how to check the convergence rate of **practical training**.

2. No matter what the theory says, you can **always apply these methods** to understand the convergence speed.

Caveats

1. The speed may change during training.

e.g. the first 10,000 iterations may show a $1/T$ rate, and the next 10,000 a $1/T^{0.7}$ rate.

2. In practice, F^* is unknown.

So we measure $\|\nabla F(w)\|$ instead of $F(w) - F^*$.

Without the whole dataset, $\nabla F(w)$ is only estimated, so the rate may be inaccurate.

Part II Roadmap

II.1	**Convergence Theory**	stationary points · diminishing vs. constant stepsize · two zones
II.2	**Reading Convergence Rates**	iteration complexity · the 10× test · experiments
II.3	**Why Is SGD Fast?**	cost saving · newer information · memory

A Missing Message

- Bottou et al. 2016 “we omit these complications ... since ... they do not provide significant additional insights into the forces driving convergence of the method” (talking about a different algorithm)

- Theoretical results on SGD usually **miss an important message**.
- They show why SGD is worse than GD, but not **why SGD is better**
  - SGD is widely used not because $1/\epsilon$ rate or $O(\alpha)$ error...

- but because **SGD is faster than GD**, in certain sense...

Explanation 1: Why SGD is Fast

- Bottou provided a very simple yet intuitive example.
- **Example 1:** Consider linear regression $\min_w F(w)=\sum_{i=1}^{n} 1/n f_i(w)$, where $f_i(w)=x_i^T w-y_i$.
- **Example 1a:** $x_1=x_2=\ldots=x_n$, $y_1=\ldots=y_n$, i.e., all data are identical. Then $F(w)=f_i(w)$.

- GD: $w_{new}\leftarrow w-\alpha\nabla F(w)$.

- SGD: $w_{new}\leftarrow w-\gamma\nabla f_i(w)=w-\gamma\nabla F(w)$.

- So one iteration of SGD is **equivalent** to one iteration of GD if $\gamma=\alpha$
- **Difference**: the cost of SGD is $1/n$-th of GD

Claim: For Example 1a, SGD is n times faster than GD.

Implication: Correlation is Good for SGD

How about general case?

GD: $w_{new} \leftarrow w - \alpha \nabla F(w)$.

SGD: $w_{new} \leftarrow w - \gamma \nabla f_i(w) \neq w - \gamma \nabla F(w)$.

Note that $f_i(w)$ is a **stochastic estimate** of $F(w)$; thus in long term roughly similar effect as $F(w)$ (but worse than using $\nabla F(w)$).

Nevertheless, each iteration is $1/n$ **cheaper**.

Battle between two effects: cost saving v.s. harm of inexact gradient.

In “fast zone”, “cost saving” effect **always wins**! Thus SGD is faster than GD (for low accuracy)

Implication on Correlation?

Correlation between mini-batches larger ⇒ faster

- **Practical tip**: Shuffle data, s.t., similar data are not in same mini-batch
- **Example**: binary classification, let all mini-batches have similar components (often implies mixture of positive/negative samples)
- Suppose data are (dog, dog, ..., dog, cat, cat, ..., cat), half dogs, half cats.
- **Partition 1**: (dog, ..., dog), ..., (dog, ..., dog), (cat, ..., cat), ..., (cat, ..., cat)
- **Partition 2**: (dog, ..., dog, cat, ..., cat), (dog, ..., dog, cat, ..., cat), ..., (dog, ..., dog, cat, ..., cat)
- **Partition 2** is likely to have higher correlation than **Partition 1**, so choose **Partition 2**

- Remark: In Partition 1, dog-group and dog-group have high correlation, but consider all pairs, since dog-group and cat-group has low correlation, the overall correlation is relatively low

Explanation 2: Why SGD is Fast

Example 1b: $n=2$.

- SGD: $w_{temp} \leftarrow w - \gamma \nabla f_1(w)$,
  $w_{new} \leftarrow w_{temp} - \gamma \nabla f_2(w_{temp})$,
- SGD one epoch:
  $$w_{new} \leftarrow w - \gamma [\nabla f_1(w) + \nabla f_2(w_{temp})].$$

- GD:
  $$w_{new} \leftarrow w - \alpha [\nabla f_1(w) + \nabla f_2(w)].$$

Q1: Why is SGD faster than GD based on this comparison?

Q2: How is this explanation related to Explanation 1?

Explanation 2: Why SGD Is Fast — Answers

One epoch of SGD and GD:

- SGD: $w_{new} \leftarrow w - \gamma[\nabla f_1(w) + \nabla f_2(w_{temp})]$.
- GD: $w_{new} \leftarrow w - \alpha[\nabla f_1(w) + \nabla f_2(w)]$.

Q1: Why is SGD faster than GD here?

SGD computes the second gradient at w_{temp}, a point it has already moved to.

This is newer information than GD uses, so SGD should move faster.

Q2: How does this relate to Explanation 1?

Example 1a: SGD saves cost along the same (or a similar) direction.

Example 1b: at the same cost (one epoch), SGD uses newer information.

So SGD gets a better per-epoch direction than GD.

Summary of Two Explanations

1 Explanation 1

Same direction → compare the cost

- **Per iteration**: SGD direction ≈ GD direction
- **Cost**: only $1/n$ of GD

Example: identical data (Example 1a)

Cost saving beats the noise → SGD is faster

2 Explanation 2

Same cost → compare the direction

- **Per epoch**: same cost as one GD step
- **Direction**: SGD uses newer information

Example: n = 2, one epoch (Example 1b)

Better per-epoch direction → SGD is faster

Why SGD? Memory Constraint

Besides converging faster, a major reason for using SGD: **memory constraint**.

ImageNet

10^6

images, each 224 × 224

→

One GPU (2080Ti, 11 GB)

≈ 64

images fit (ResNet);
in practice batch 16–32

→

Full-batch GD would need

≈ 15,625

GPUs (10^6 / 64)
≈ 1 billion Yuan

So: we cannot even compute the full gradient — mini-batch SGD is a necessity, not just a speed-up.

Large-scale Distributed Training: ImageNet + ResNet-50

	GPUs	mini-batch size	training time
He et al. 2016	8	256	29 hours
Goyal et al. 2017	256	8,192	**1 hour**
Yamazaki et al. 2019	2,048 (V100)	81,920	**≈ 1.2 minutes**

Goyal et al.: 32× more GPUs, 29× faster.
Scaling efficiency 29/32 ≈ 0.91 — remarkable.

Two optimization "tricks" (→ Part III):

- **scale the learning rate** with the mini-batch size; **gradual warmup**

Summary of SGD

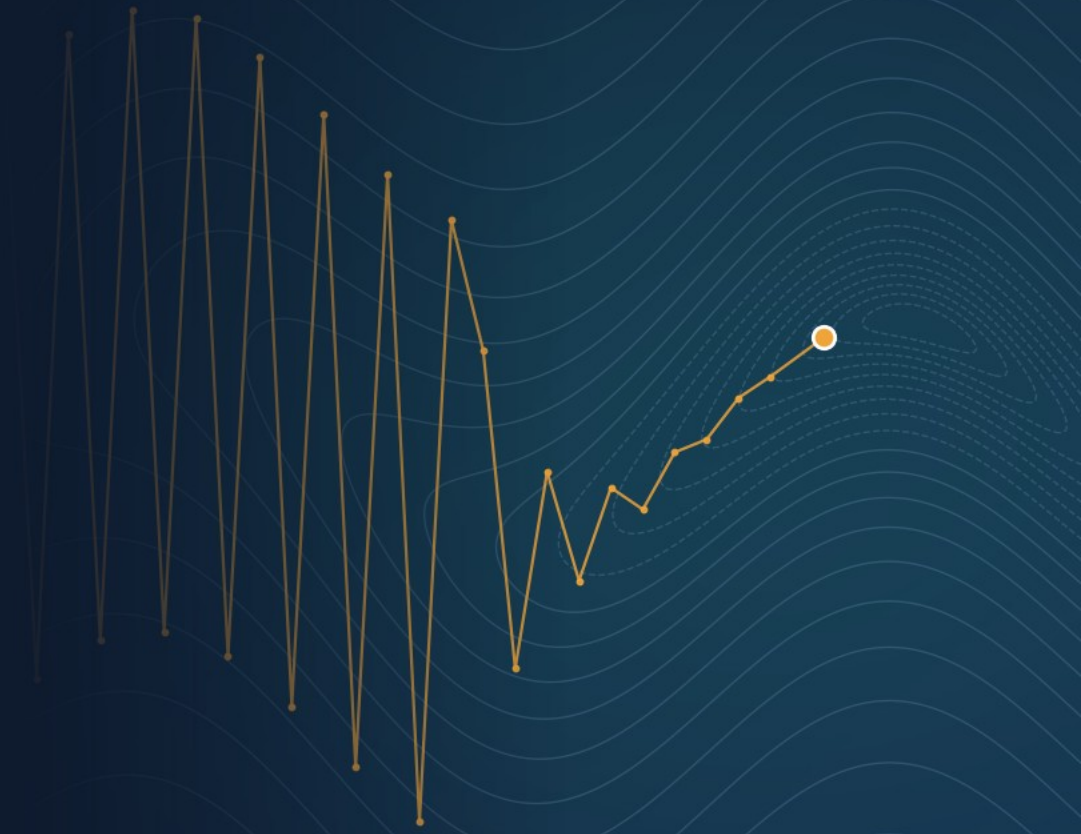
1. Different variants of SGD: mini-batch, shuffled/randomized
2. Convergence: two zones for constant stepsize; diminishing stepsize (square summable) + L-smoothness + bounded variance
3. Various lr schedules in DL libraries → Part III
4. Convergence speed: fewer FLOPs than GD (for low accuracy), in sequential model
5. **Key to understand benefit of SGD: limited memory**

Next: how to set the stepsize in practice → learning rate schedules (Part III); then from SGD to Adam (Part IV)

PART III

Learning Rate Schedules

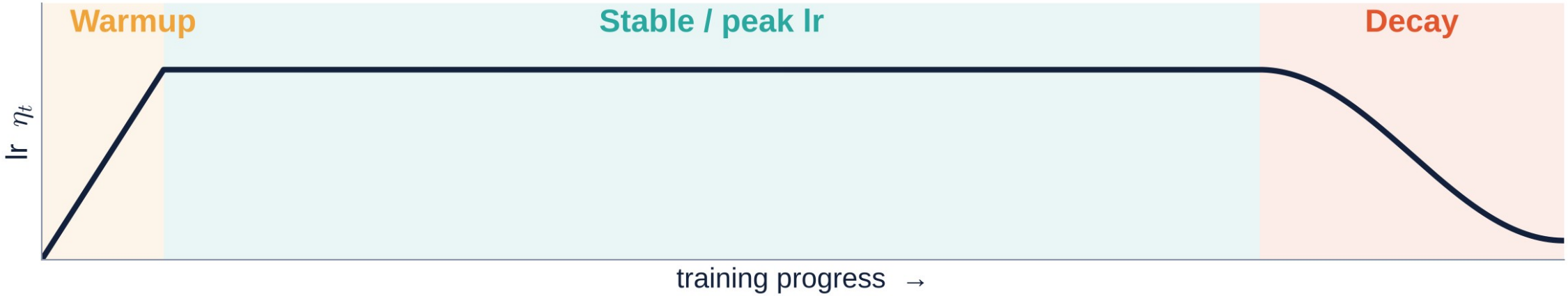
Step, warmup, cosine, WSD …
One of the most important — and most overlooked — hyperparameters in LLM training



I SGD & Variants II SGD Theory III LR Schedules IV From SGD to Adam

A Popular Learning Rate Schedule Today

$\theta_{t+1} = \theta_t - \eta_t \cdot u_t$ direction u_t from SGD / Adam / Muon; the schedule sets η_t



This warmup → stable → decay shape is a **popular learning rate schedule today**.

1. **First: understand the steps**
   warmup, stable phase, decay — one at a time
2. **Then: the bigger picture**
   other learning rate schedules used over the years

Schedule 1: Constant and Step Decay

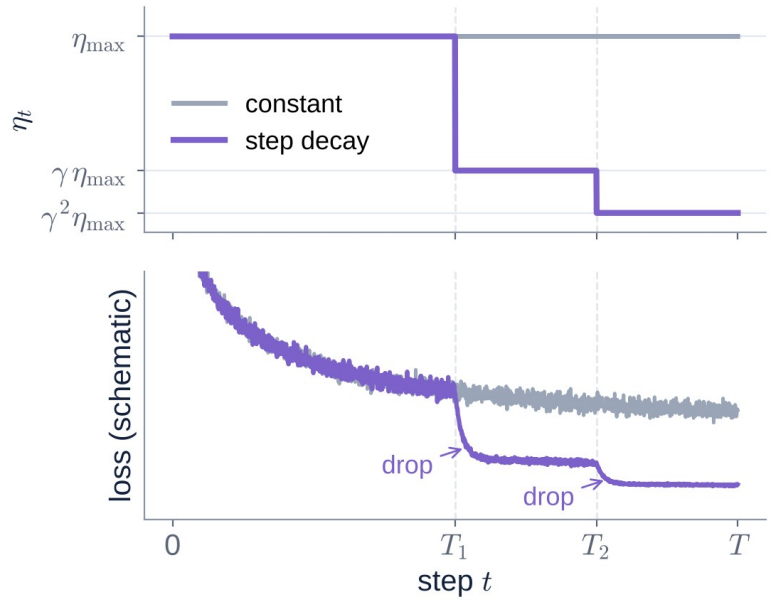
$$\eta_t = \begin{cases} \eta_{max}, & 0 \le t < T_1; \\ \gamma\eta_{max}, & T_1 \le t < T_2; \\ \gamma^2\eta_{max}, & t \ge T_2 \end{cases}$$

Constant: $\eta_t = \eta_{max}$ for all t (gray).
Step decay: $\times\gamma$ at each milestone (purple, $\gamma \approx 0.32$).

Used by
Constant: fine-tuning
ResNet (2016): 0.1, ÷10 at plateaus
DeepSeek LLM (2024): ×0.316 at 80%, 90%

Each drop → a sudden loss drop.

✗ **Cons:** milestones and γ chosen by hand



Remark 1: two drops is just an example; the number varies.
Remark 2: when to drop?
• fixed milestones, e.g. PyTorch `MultiStepLR`;
• when the loss stops improving: `ReduceLROnPlateau`.

Schedule 2: Warmup

Why warm up? Training is unstable at the start

1. Local Lipschitz constants (max Hessian eigenvalue) may be **large in early iterations** → use a smaller lr
2. A small lr first lets the model reach a **smoother region** where a large lr is stable [Gilmer et al., 2022]
3. Post-LN Transformers diverge without warmup [Xiong et al., 2020]

In practice

- Linear: $\eta_t = \eta_max \cdot t / T_w$
- T_w: often 1–5% of training; LLMs: a few hundred to a few thousand steps (often < 1%)



2000 steps	**2K** steps	**500** steps
Llama 2	**DeepSeek-V3**	**Kimi K2**

Remark: after warmup the lr is not flat — cosine / WSD decay (discussed later).

Schedule 3: Inverse Square Root

Transformer (2017): linear warmup, then decay

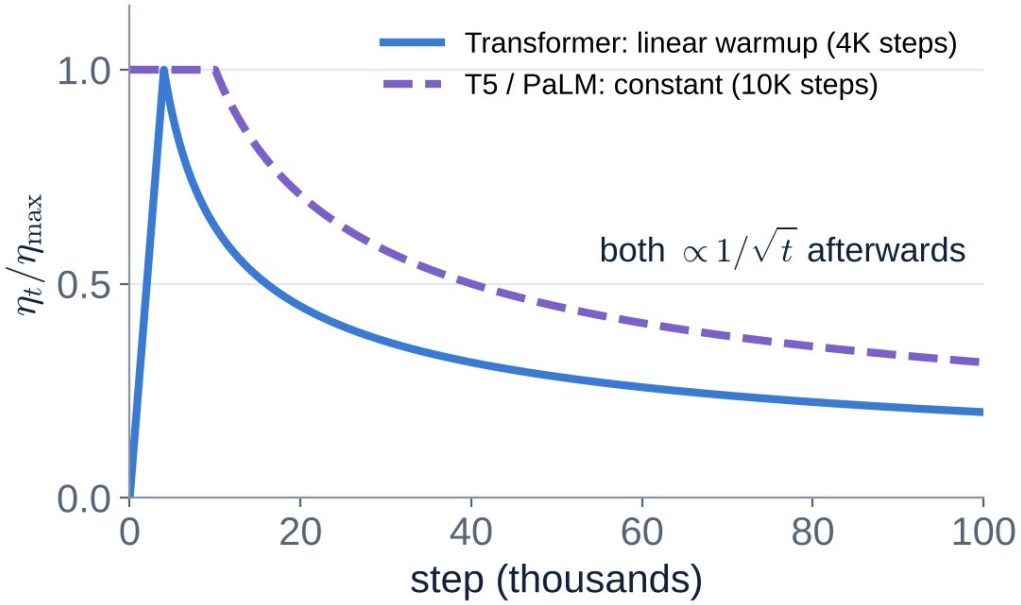
$\eta_t = \eta_{max} \cdot \min(t/T_w, \sqrt{(T_w/t)}), \quad T_w = 4000$

T5 (2020), PaLM (2022): constant, then decay

$\eta_t = \eta_{max} \cdot \sqrt{(T_w/(\max(t, T_w)))}, \quad T_w = 10^4$

After T_w, both give $\eta_t \propto 1/\sqrt{t}$:
This **matches the theory** (Part II.1).

Who used it? Rarely used before LLMs.
Google's early LMs: Transformer, T5, PaLM.



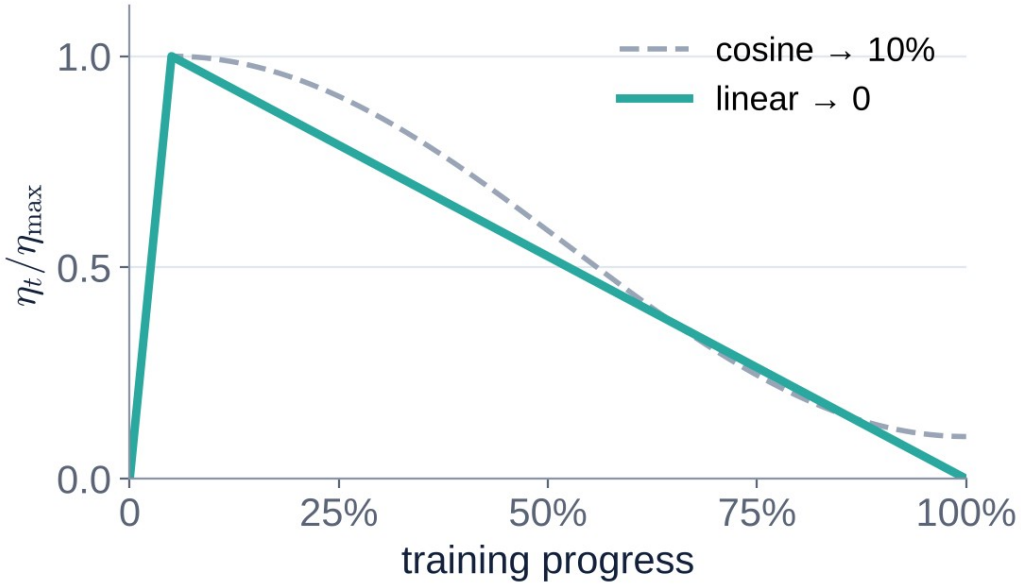
Schedule 4: Linear Decay

$$\eta_t = \eta_{max} \cdot (1 - t / T)$$

Compared to $1/\sqrt{t}$:
directly use **linear decay**.
It goes all the way to 0 — no slow tail.

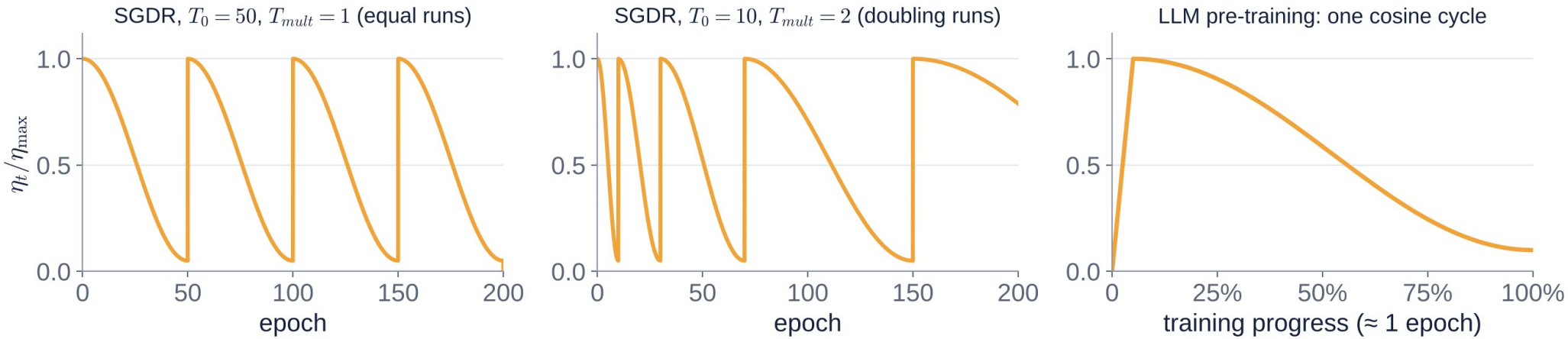
Used by: BERT (2019): warmup, then linear to 0

✗ **Cons:** needs the total steps T in advance.



Schedule 5: Cosine — the Classic Default

$$\eta_t = \eta_{min} + \tfrac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\pi t / T))$$



Originally: SGDR [Loshchilov & Hutter, 2017]
Each run: cosine from η_{max} to η_{min}.
Restart: lr jumps back to η_{max}; runs equal or doubling.

LLM pre-training: one cycle
GPT-3: cosine to 10% over 260B of 300B tokens.
Llama 2: 2000-step warmup, cosine to 10%.

Schedule 6: WSD (Warmup – Stable – Decay)

Recap: cosine decay (previous slide)

In LLMs, “cosine” usually means **warmup + cosine decay right from the peak**.

Another schedule: WSD (two variants)

WSD

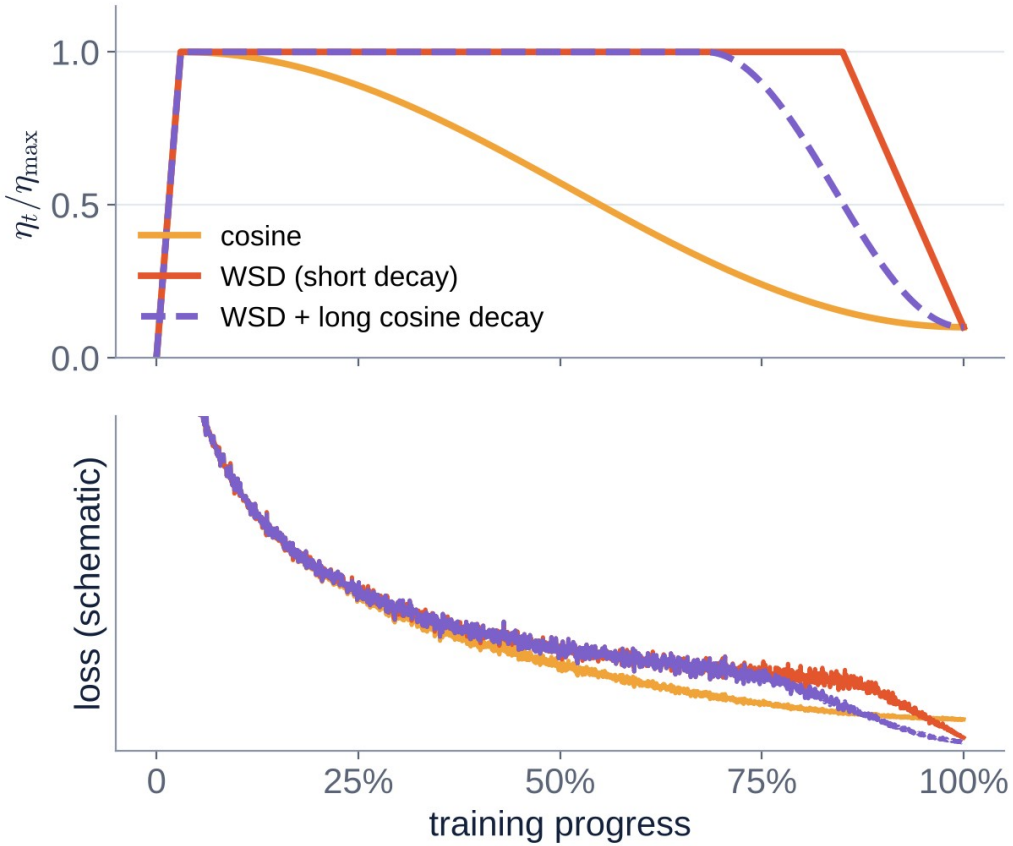
warmup → **constant** → **short decay** (last ~10–25%)

decay: exponential (MiniCPM), linear, or $1 - \sqrt{(t/T_d)}$ (Nemotron 3)

WSD with a long cosine decay

constant for ~⅔ → **cosine decay** over the last ~⅓

DeepSeek-V3/V4, Kimi K2, MiMo-V2 ("WSD-like")



New Trends: Schedule-Free and Weight Averaging

1

Schedule-Free

Defazio et al., NeurIPS 2024

Replaces lr decay by iterate averaging — no need to fix T.

Won the AlgoPerf 2024 self-tuning track.

2

Weight averaging

e.g. WSM, 2025

Averaging checkpoints from the stable phase (EMA / merging) can replace the decay phase.

Common intuition: decay ≈ reducing noise ≈ averaging the iterates.

In Practice: Which Schedule Did LLMs Use (2020–2025)?

Model	Released	Warmup	After warmup	Type
GPT-3	2020.05	375M tokens	cosine to 10% over 260B of 300B tokens	cosine
Llama 2	2023.07	2000 steps	cosine to 10% of the peak	cosine
Llama 3 405B	2024.07	8000 steps	cosine over 1.2M steps; anneal to 0 at the end	cosine
DeepSeek-V3	2024.12	2K steps	constant to 10T tokens → cosine (4.3T)	WSD-like
Kimi K2	2025.07	500 steps	constant for 10T tokens → cosine (5.5T)	WSD-like
GLM-4.5	2025.08	✓	cosine (WSD underfit in their tests)	cosine

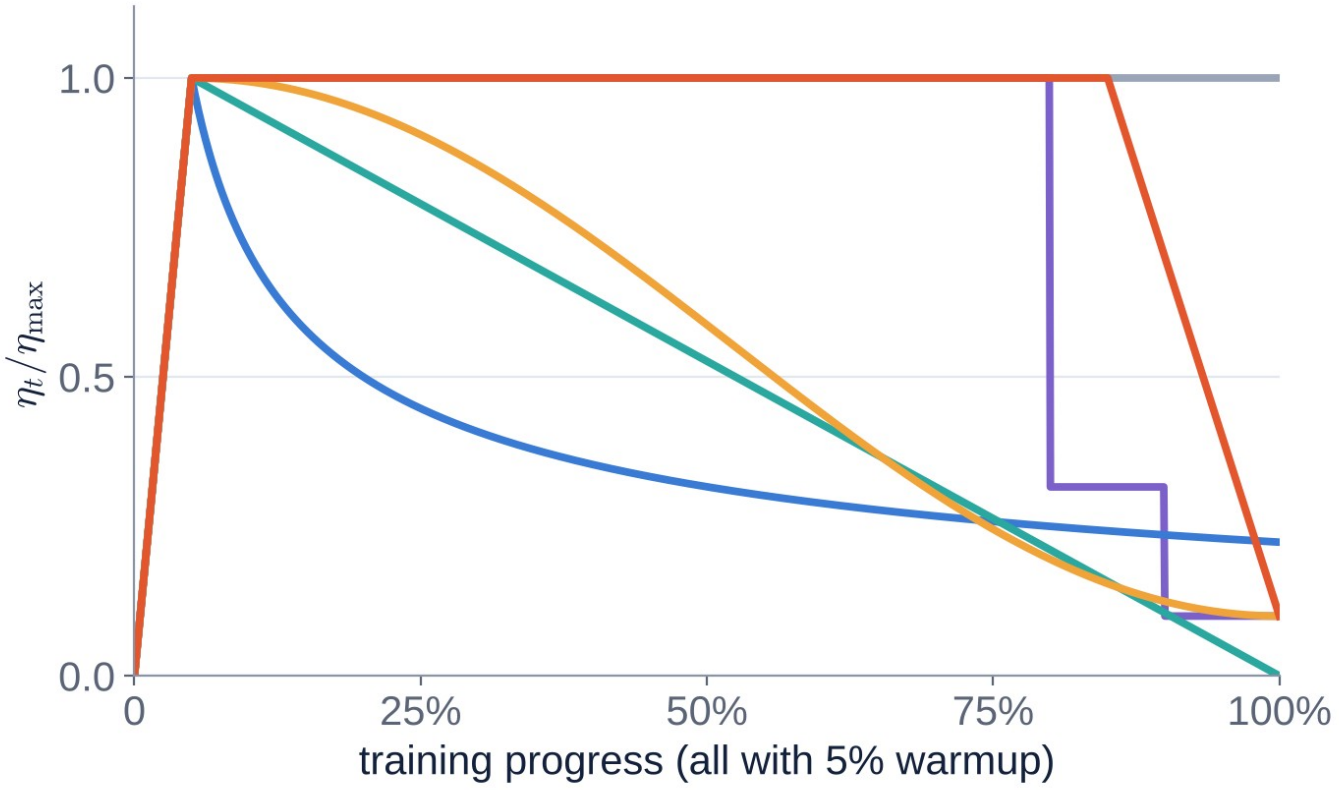
Pattern: cosine was the classic default; from 2024, some labs switched to a long constant phase + a final decay (WSD-like).

The Newest Open Models (2026): Cosine or WSD?

Model	Released	Schedule (pre-training)	Type
MiMo-V2-Flash	2026.01	constant for 12T tokens → cosine (10T + 4T)	**WSD-like**
Step 3.5 Flash	2026.02	cosine over 14.6T tokens → short anneal	**cosine**
Nemotron 3 Ultra	2026.06	stable to 15T → $1 - \sqrt{(t/T_d)}$ decay (last 5T)	**WSD**
Kimi K3	2026.07	1% warmup → cosine; beat WSD in their scaling-law study	**cosine**
DeepSeek-V4.1-Flash	2026.09	peak to 28T → cosine to 10% (40T) → stays at 10% to 45T	**WSD-like**
Qwen3.8-Max	2026.08	*no technical report — not disclosed*	?
GLM-5.3	2026	*post-training only (base = GLM-5.2) — not disclosed*	?
Meta Muse Spark Open	2026.10	*open weights, no training report — not disclosed*	?

No single winner in 2026: WSD (Nemotron 3), WSD-like (DeepSeek, MiMo) and cosine (Kimi K3, Step) are all used.

Summary: Common Learning Rate Schedules



Constant
early work; fine-tuning

Multi-step
ResNet; DeepSeek LLM

Inverse sqrt
Transformer, T5, PaLM

Linear → 0
BERT

Cosine
GPT-3, Llama 2/3, GLM-4.5, Kimi K3

WSD / WSD-like
MiniCPM, Nemotron 3; DeepSeek, Kimi K2, MiMo

Summary: How η Changes with the Step t

Schedule	Formula (T: total steps, Tw: warmup steps)	Typical use
Linear warmup	$\eta_t = \eta_{max} \cdot t / T_w, \quad t \le T_w$	almost all Transformers
Step decay	$\eta_t = \gamma^k \eta_{max}$, k = milestones passed	ResNet; DeepSeek LLM
Inverse sqrt	$\eta_t = \eta_{max} \cdot \sqrt{(T_w/t)}, \quad t \ge T_w$	Transformer, T5, PaLM
Linear decay	$\eta_t = \eta_{max} \cdot (1 - t / T)$	BERT
Cosine	$\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\pi t/T))$	GPT-3, Llama 2/3, GLM-4.5, Step 3.5, Kimi K3
WSD	warmup → constant η_{max} → short decay at the end	MiniCPM, Nemotron 3
WSD-like	warmup → constant η_{max} → long cosine decay	DeepSeek-V3/V4, Kimi K2, MiMo-V2

Lab: LR Schedule Lab

Train the same network on MNIST with constant, step, cosine and WSD schedules under identical conditions, and compare loss and test accuracy.

Open the lab →

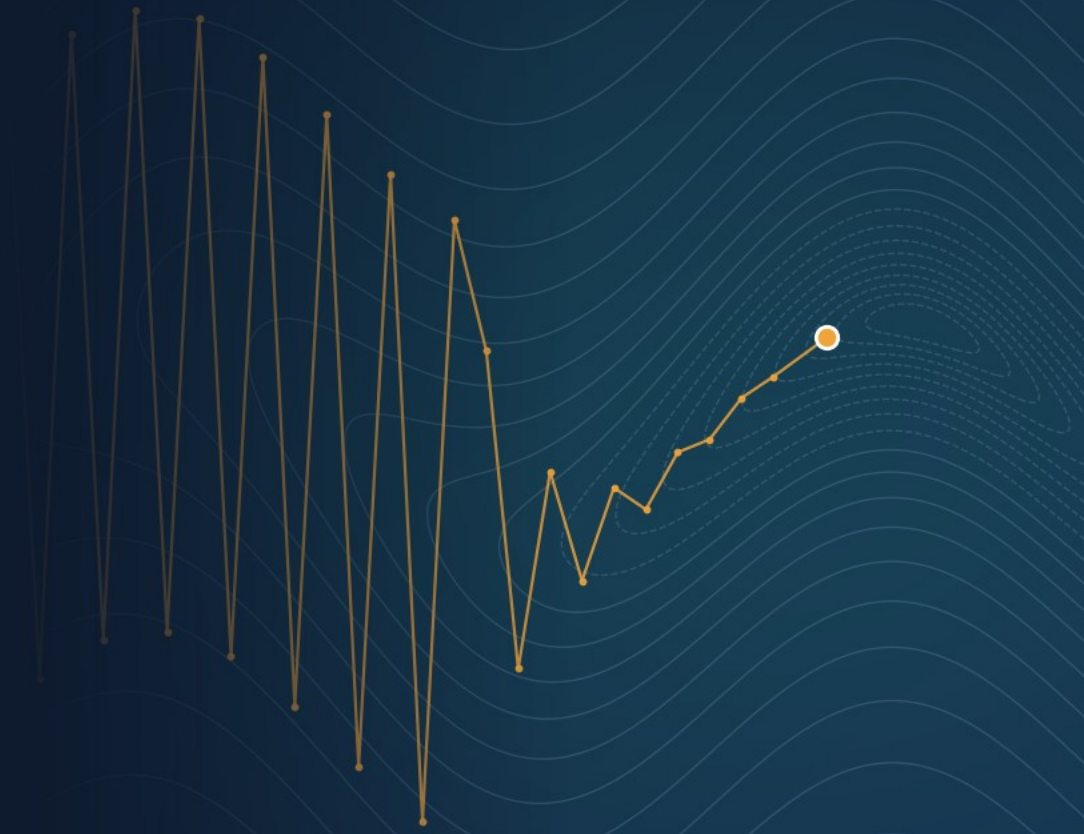
https://ruoyus.github.io/me2605/demos/lr-schedule-lab/

scan to open on your phone

PART IV

From SGD to Adam

Averaging, momentum and adaptive stepsizes: AdaGrad → RMSProp → Adam



I SGD & Variants II SGD Theory III LR Schedules IV From SGD to Adam

Background

Adam is the most popular algorithms in deep learning (DL). (**210k** citations, by March 2024)

Adam is probably the single **most-cited** optimization paper in history

```
optimizer = optim.Adam(net.parameters(), lr=args.lr, betas=(args.beta1, args.beta2), eps=1e-08,
                       weight_decay=args.weightdecay, amsgrad=False)
```

Adam is, for industrial LLM training, **one of the main choices**

(Muon **emerges recently; will discuss later**)

Averaging Trick on Weights

Averaging (or summing) is common in algorithms.

Example 1: Polyak-Ruppert averaging (1988,1990); averaged SGD (ASGD).
Apply averaging to weights (0th order info).

$$\textbf{Regular update}: w_{t+1} = w_t - \alpha \nabla F_{i_t}(w_t), t = 0, 1, \ldots.$$

$$\textbf{Averaging}: \quad \bar{w}_{t+1} = \frac{t}{t+1}\bar{w}_t + \frac{1}{t+1} w_{t+1}, t = 0, 1, \ldots. \tag{1}$$

A variant called EMA is also popular.

$$\textbf{Regular update}: w_{t+1} = w_t - \alpha \nabla F_{i_t}(w_t),$$

$$\textbf{EMA}: \quad \bar{w}_{t+1} = \beta \bar{w}_t + (1-\beta) w_{t+1}. \tag{2}$$

Claim 1a: (1) implies $\bar{w}_t = \frac{1}{t}\sum_{i=1}^{t} w_i, t = 1, 2, \ldots.$
Claim 1b: (2) and $\bar{w}_{-1} = 0$ implies $\bar{w}_t = (1-\beta)\sum_{i=0}^{t} \beta^{t-i} w_i.$

PyTorch implementation of ASGD: torch.optim.ASGD

Averaging Trick on Gradients: Momentum

Example 2: SGD with Momentum. Let $g_t = F_{i_t}(w_t)$.

$$v_t = \beta v_{t-1} - \alpha \nabla F(\mathbf{w}_t), \quad t = 0, 1, \ldots. \tag{3}$$
$$\mathbf{w}_{t+1} = \mathbf{w}_t + v_t, \quad t = 0, 1, \ldots.$$

Claim 2: (3) implies

$$v_t = \beta^{t+1} v_{-1} - \alpha \sum_{k=0}^{t} \beta^{t-i} g_k. \tag{4}$$

When $v_{-1} = 0$, the above is the exponential moving average of the gradients (first-order info) $(\nabla F(w_0), \ldots, \nabla F(w_t))$.

AdaGrad

Let $\circ$ denote entry-wise product. For a vector u, the notation $\sqrt{u}$ and $1/u$ also means entry-wise square root and entry-wise reciprocal.

At the t-th iteration, randomly pick i, and update the parameter as

$$\begin{aligned} g_t &= \nabla F_i(\theta_t), \\ v_t &= v_{t-1} + g_t \circ g_t, \\ \theta_{t+1} &= \theta_t - \alpha_t \frac{g_t}{\sqrt{v_t} + \epsilon}, \quad t = 0, 1, 2, \ldots, \end{aligned} \tag{5}$$

In other words, the step-size for the k-th coordinate is adjusted from α_t in standard SGD to $\alpha_t / \sqrt{\sum_{j=0}^{t} g_{j,k}^2}$ where $g_{j,k}$ denotes the k-th entry of g_j.

Effectively, $\theta_{t+1} = \theta_t - \alpha_t (\sum_{j=0}^{t} g_j \circ g_j)^{-1/2} \circ g_t$.

Diagonal preconditioner!

RMSProp

One drawback of AdaGrad: it treats all samples equally.

Natural idea: use EMA to define v_t.

RMSProp: At the t-th iteration, randomly pick i and compute $g_t = \nabla F_i(\theta_t)$, then:

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t \circ g_t,$$
$$\theta_{t+1} = \theta_t - \alpha_t \frac{g_t}{\sqrt{v_t} + \epsilon}.$$

PyTorch default: $\beta_2 = 0.99, \epsilon = 10^{-8}$ (together with $\alpha_t = 0.01$)
Common choice: $\beta_2 \in \{0.9, 0.99, 0.999\}$.

Adam

Adam [Kingma and Ba, 2014] is (almost) the combination of RMSProp and momentum.

Adam: At the t-th iteration of Adam, we randomly pick i and compute $g_t = \nabla F_i(\theta_t)$, then:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad \text{[update 1st order momentum]}$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t \circ g_t, \qquad \text{[update “2nd order” momentum]}$$
$$\theta_{t+1} = \theta_t - \alpha_t \frac{m_t}{\sqrt{v_t} + \epsilon} \qquad \text{[update parameter]} \ .$$

PyTorch default: $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$ (together with $\alpha_t = 0.001$).

learning rate: α_t can be constant, or follow a certain LR schedule.

Exact Version of Adam: With Bias-correction Term

[Kingma, Ba'15] added bias-correction term:

$m_0 \leftarrow 0$ (Initialize 1st moment vector)
$v_0 \leftarrow 0$ (Initialize 2nd moment vector)
$t \leftarrow 0$ (Initialize timestep)
while θ_t not converged **do**
  $t \leftarrow t + 1$
  $g_t \leftarrow \nabla_\theta f_t(\theta_{t-1})$ (Get gradients w.r.t. stochastic objective at timestep t)
  $m_t \leftarrow \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ (Update biased first moment estimate)
  $v_t \leftarrow \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ (Update biased second raw moment estimate)
  $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ (Compute bias-corrected first moment estimate)
  $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ (Compute bias-corrected second raw moment estimate)
  $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ (Update parameters)
end while
return θ_t (Resulting parameters)

Useful for the first few epochs; in theory, not affecting later convergence.

- For simplicity, we use (8), i.e., without bias-correction.

Algorithms in PyTorch

Algorithms in torch.optim can be divided into three classes.

First, **SGD** (including momentum).

- SGD
- ASGD

Second, **LBFGS.** (famous nonlinear optimization method).

Third, **adaptive gradient methods**.

- Adagrad, RMSProp, Adam (including AMSGrad), Adadelta
- AdamW, Adamax, SparseAdam,

Adam Variants

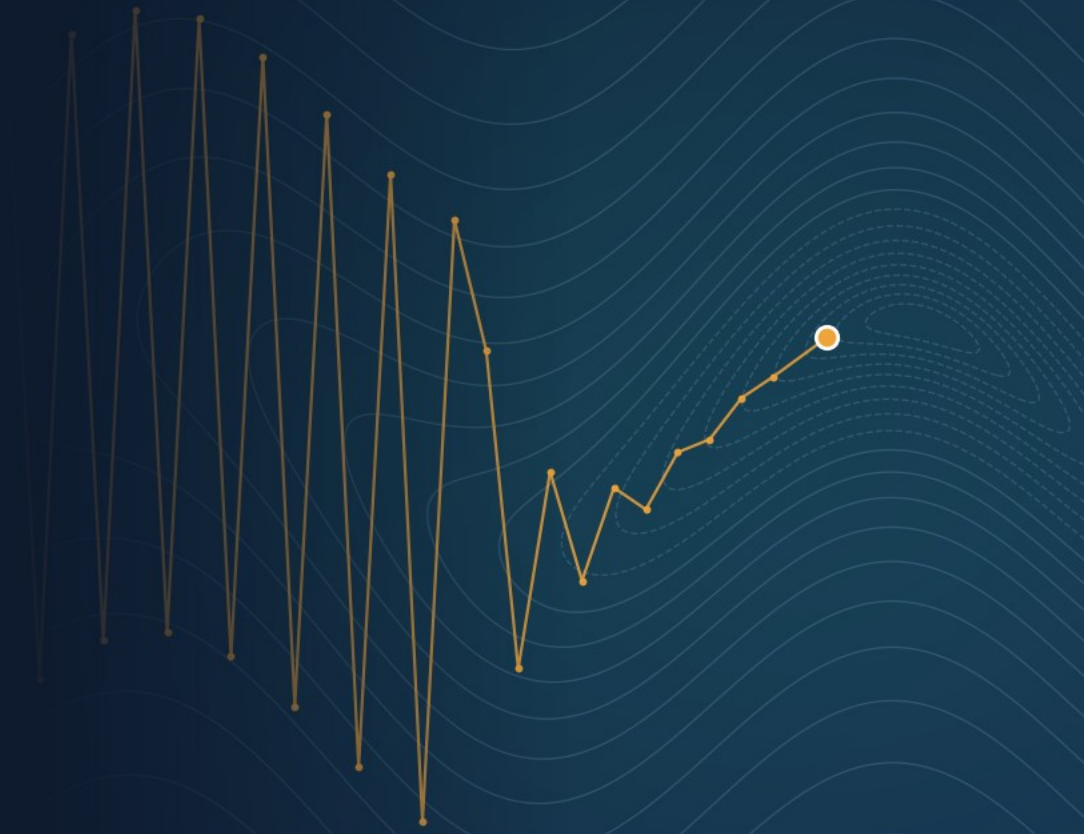
Brief intro of these adaptive gradient methods:

- Adadelta: related to RMSProp, but more complicated numerator
- AdamW: “correct” way to implement weight decay and Adam together.
- Adamax: a variant of Adam based on infinity norm
- SparseAdam: Implements lazy version of Adam algorithm suitable for sparse tensors.
- Adam-mini: use block-wise mean to replace coordinate-wise lr, Zhang et al., 2024.

SUMMARY

Summary

SGD and its variants, SGD theory, LR schedules, from SGD to Adam —
what to take away



SUMMARY

What to Take Away — Parts I and II

PART I
SGD & Variants

- Each step uses one sample or a mini-batch, not all n samples.
- In practice: random reshuffling every epoch + mini-batch + momentum.

PART II.1
Convergence Theory

- Constant α: a fast zone, then a **confusion zone** of size $\propto \alpha$.
- Diminishing $\alpha_t = \alpha_0/\sqrt{t}$: converges at rate $O((\log T)/\sqrt{T})$ (nonconvex).

PART II.2
Reading Rates

- Error $O(1/T) \Leftrightarrow 1/\varepsilon$ iterations; error $O(1/\sqrt{T}) \Leftrightarrow 1/\varepsilon^2$ iterations.
- The 10× test: count the steps needed to cut the error by 10×.

PART II.3
Why SGD Is Fast

- Explanation 1: about the same direction as GD at $1/n$ of the cost.
- Explanation 2: same cost per epoch, but each step uses newer information.
- Memory: one GPU holds only a few samples, so full-batch GD is impractical.

SUMMARY

What to Take Away — Parts III and IV

PART III
LR Schedules

- Today's popular shape: linear warmup → (stable) → decay.
- Classic decays: step, inverse sqrt (matches the theory), linear, cosine.
- LLMs: **cosine** (GPT-3, Llama, Kimi K3) or **WSD-like** (DeepSeek, MiMo).
- Warmup: a short linear ramp (often < 1% of the steps) stabilizes the start.

PART IV
From SGD to Adam

- Averaging: on weights (Polyak averaging, EMA) and on gradients (momentum).
- Per-coordinate stepsize: AdaGrad → RMSProp (EMA of squared gradients).
- Adam = momentum + RMSProp (+ bias correction); variants such as AdamW.

SUMMARY

Check Yourself

01 Name the three design choices in SGD, and say what each one changes.

02 State when SGD converges, for diminishing and for constant stepsizes.

03 Tell which rate a run follows, from the time it takes to cut the error 10×.

04 Explain why SGD is fast, and when memory forces the batch size.

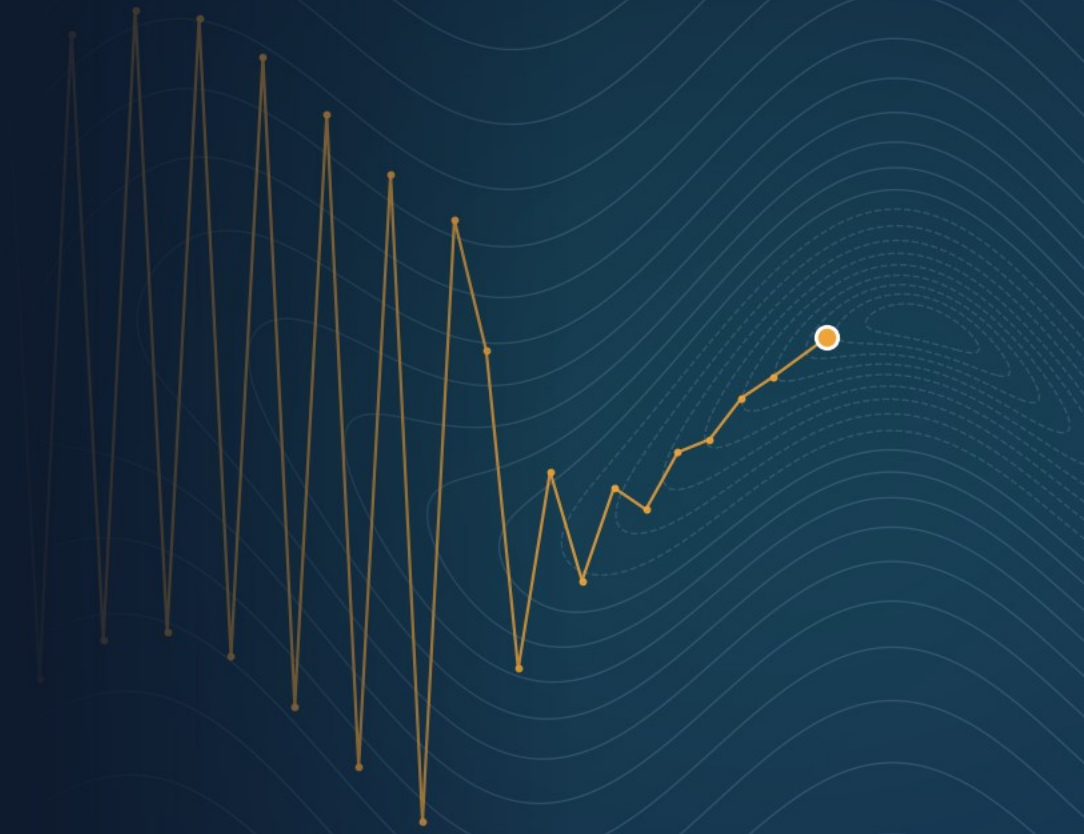
05 Choose a learning-rate schedule, and say what each one changes.

06 Derive Adam from momentum and a diagonal preconditioner.

END

Questions

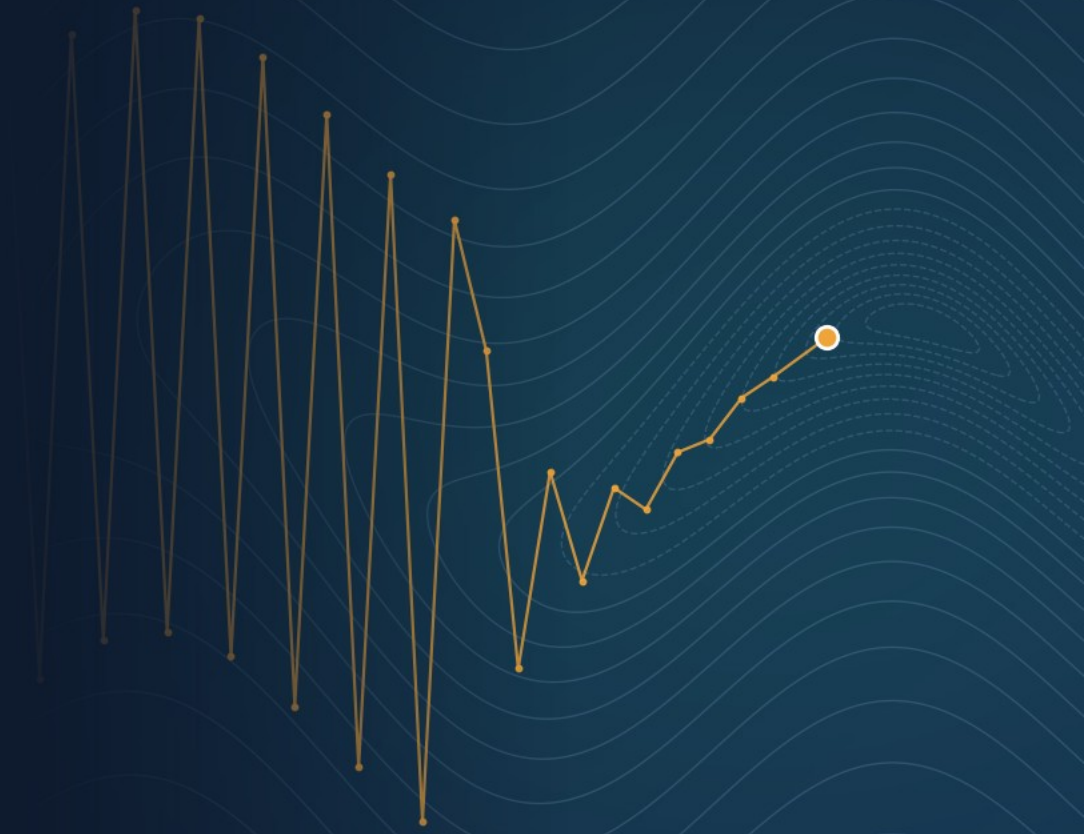
Thank you.
Slides and lab links are posted with the course material.



APPENDIX

Appendix

Proof of Theorem 2 (constant stepsize, strongly convex)



Proof of Theorem 2 (Constant Stepsize, SC)

Theorem 2.1.11 in [Nesterov, 2013] states that

$$\langle w-w^*,\nabla f(w)\rangle \geq \mu L/(\mu+L)\|w-w^*\|^2+1/(\mu+L)\|\nabla f(w)\|^2. \qquad (3)$$

Denote $g_k=\nabla F(w_k)$. Recall $w_{k+1}=w_k-\alpha\nabla F(w_k;\xi_k)$. Conditioned on w_k, we have:

$$[E\|w_{k+1}-w^*\|^2 =^{(i)} \|w_k-\alpha g_k-w^*\|^2+\alpha^2\sigma^2; \leq^{(ii)} (1-2\alpha\mu L/(\mu+L))\|w_k-w^*\|^2+\alpha(\alpha-2/(\mu+L))\|g_k\|^2+\alpha^2\sigma^2;$$
$$\leq^{(iii)} (1-2\alpha\mu L/(\mu+L))\|w_k-w^*\|^2+\alpha(\alpha-2/(\mu+L))\mu^2\|w_k-w^*\|^2+\alpha^2\sigma^2; \leq^{(iv)} (1-\mu\alpha)^2\|w_k-w^*\|^2+\alpha^2\sigma^2.]$$

Here (i) is due to the independence of the gradient error at the k-th iteration and the current iterate w_k. (ii) is due to (3). (iii) is due to $\alpha-2/(\mu+L)\leq 0$. (iv) is due to $\|\nabla F(w)\|\geq\mu\|w-w^*\|$ (strong convexity).

Taking expectation over $\xi_0,\ldots,\xi_k$ over both sides, we get $\Phi_{k+1}\leq(1-\mu\alpha)^2\Phi_k+\alpha^2\sigma^2$. This implies $\Phi_{k+1}-\Phi^*\leq(1-\mu\alpha)^2[\Phi_k+\alpha^2\sigma^2-\Phi^*]$, where $\Phi^*=(\alpha\sigma^2)/(\mu(2+\mu\alpha))$.